Del Balance al Dashboard: Construcción de Indicadores Financieros desde Cero

Toda la parte de Python del proyecto, corregida y explicada paso a paso.

Dataset: Financial Data of 4400+ Public Companies (Kaggle): balance, cuenta de resultados y flujo de caja anuales de 4.422 empresas cotizadas, 2012-2020.

La idea del proyecto

La mayoría de análisis financieros parten de ratios ya calculados por otros. Aquí se construyen **desde los 3 estados contables crudos**. El valor no está en el número final, sino en el proceso:

- cómo se **relacionan** los tres estados,
- qué **criterio de negocio** hay detrás de cada fórmula,
- cómo se tratan los **casos límite** (patrimonio negativo, divisiones por cero, valores disparatados).

Los 3 estados contables

Estado	Qué cuenta	Ejemplos de partidas
Balance (*balance sheet*)	Lo que la empresa **tiene** y lo que **debe** en una fecha	Activo, pasivo, patrimonio neto, inventario
Cuenta de resultados (*income statement*)	Lo que **ha ganado o perdido** en el año	Ingresos, beneficio operativo, beneficio neto, intereses
Flujo de caja (*cash flow*)	El **dinero real** que ha entrado y salido	Flujo de caja operativo, inversiones (CapEx), dividendos

Qué reúne este notebook

Antes, la parte de Python estaba repartida en 9 archivos. Este notebook los reúne en uno solo, en el orden lógico:

Archivo original	Dónde está ahora
`exploracion_sql.ipynb`	Paso 3: exploración en SQL
`indicators_sqlite.py` e `indicators_mysql.py`	Pasos 2, 4, 5 y 6: carga, indicadores y valores extremos
`proyecto_indicadores_financieros.ipynb` y `..._mysql.ipynb`	Toda la narrativa de los pasos 4 a 7
`build_excel_sqlite.py` y `build_excel_mysql.py`	Paso 8: ficha en Excel
`build_charts_sqlite.py` y `build_charts_mysql.py`	Paso 9: gráficos en Excel

🔧 Había **dos versiones casi idénticas** de todo (SQLite con columnas en inglés y MySQL con columnas en español). Ahora hay **un solo código**: eliges la fuente en la configuración y el resto funciona igual, siempre con los nombres en español.

Índice

1. Configuración
2. Conexión a la base de datos (SQLite o MySQL)
3. Exploración de los datos en SQL
4. Carga y unión de los 3 estados
5. Construcción de los indicadores (liquidez, rentabilidad, solvencia y calidad del beneficio)
6. Tratamiento de valores extremos
7. Pipeline completo y ficha financiera
8. Exportación de la ficha a Excel
9. Gráficos nativos en el Excel
10. Conclusiones

> **Errores corregidos respecto a la versión anterior** (marcados con 🔧 a lo largo del notebook):
>
> - Los notebooks de SQLite importaban `indicators`, pero el archivo se llama `indicators_sqlite.py` → **no se podían ejecutar**.
> - Los scripts de Excel guardaban en `/mnt/user-data/outputs/`, una ruta que **no existe en tu ordenador**.
> - En el paso a paso, los **valores extremos se marcaban antes de quitar los infinitos**, por eso salían tablas con `inf` y la cobertura de intereses con un 0 % de extremos.
> - La **media del ROE de 2018 salía 468** (un 46.810 %) en la exploración SQL, sin comentarlo.
> - **Google aparecía dos veces** (GOOG y GOOGL) en los rankings, y como él otros cierres duplicados.
> - Al **importar los CSV a MySQL**, las celdas vacías se convirtieron en **ceros**: 2.245 flujos de caja libre inflados en la ficha final.
> - El gráfico de **evolución de la rentabilidad** usaba medias que daban un margen neto medio de −190 %.
> - La fórmula de la cobertura de intereses **en los textos** no indicaba el valor absoluto que sí usa el código.
> - Los valores vacíos se escribían en el Excel como `nan`.
> - La contraseña de MySQL se escribía **dentro del código**; ahora se pide al ejecutar y no queda guardada.

1. Configuración

Todo lo que puede cambiar de un ordenador a otro está aquí:

- `FUENTE` : `'sqlite'` (archivo `financial_data.db` en la carpeta del notebook) o `'mysql'` (base `financial_data` en MySQL Workbench).
- Las rutas de salida son **relativas**: el Excel se guarda en la carpeta `output/`, junto al notebook. 🔧 Antes era `/mnt/user-data/outputs/`, la carpeta de mi entorno, y en tu Windows fallaba al guardar.

```
# Descomenta la primera vez si te falta alguna librería
# %pip install pandas numpy matplotlib openpyxl sqlalchemy pymysql
```

```
import os
import pathlib
from getpass import getpass

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sqlalchemy import create_engine, text

# --- Fuente de datos: 'sqlite' o 'mysql' ---
FUENTE = 'sqlite'

# SQLite
RUTA_SQLITE = pathlib.Path('financial_data.db')
CARPETA_CSV = pathlib.Path('datos')   # los 3 CSV anuales de Kaggle (para crear la base SQLite si no existe)

# MySQL (la contraseña NO se escribe aquí: se pide al conectar, ver paso 2)
MYSQL = {'host': 'localhost', 'port': 3306, 'user': 'root', 'database': 'financial_data'}

# --- Salidas ---
CARPETA_SALIDA = pathlib.Path('output')
CARPETA_SALIDA.mkdir(exist_ok=True)
RUTA_EXCEL = CARPETA_SALIDA / 'ficha_financiera_empresas.xlsx'

# --- Valores extremos: percentiles que delimitan lo "normal" ---
PERCENTIL_INF, PERCENTIL_SUP = 0.01, 0.99

pd.set_option('display.max_columns', 30)
pd.set_option('display.width', 160)
```

2. Conexión a la base de datos

2.0 Crear la base SQLite a partir de los CSV (solo la primera vez)

El dataset de Kaggle viene en **3 archivos CSV**, uno por estado contable:

CSV	Tabla que se crea	Filas
`balanceSheetHistory_annually.csv`	`balance_sheet`	17.511
`incomeStatementHistory_annually.csv`	`income_statement`	17.511
`cashflowStatement_annually.csv`	`cash_flow`	17.511

Si usas SQLite y todavía no tienes `financial_data.db`, esta celda la crea guardando cada CSV como una tabla (`to_sql`). Si ya existe, no hace nada. En MySQL, este paso lo hiciste con `schema_mysql.sql` y el *Table Data Import Wizard* de Workbench.

Pon los 3 CSV en una carpeta `datos/` junto al notebook.

```
CSV_ESTADOS = {
    'balance_sheet': 'balanceSheetHistory_annually.csv',
    'income_statement': 'incomeStatementHistory_annually.csv',
    'cash_flow': 'cashflowStatement_annually.csv',
}

if FUENTE == 'sqlite' and not RUTA_SQLITE.exists():
    import sqlite3
    with sqlite3.connect(RUTA_SQLITE) as con:
        for tabla, archivo in CSV_ESTADOS.items():
            datos = pd.read_csv(CARPETA_CSV / archivo)
            datos.to_sql(tabla, con, index=False)
            print(f'Tabla {tabla:17s} creada: {len(datos):,} filas, {datos.shape[1]} columnas')
else:
    print(f'No hace falta crear la base: FUENTE = {FUENTE!r}, {RUTA_SQLITE} existe =
{RUTA_SQLITE.exists()}')
```

```
Tabla balance_sheet     creada: 17,511 filas, 31 columnas
Tabla income_statement  creada: 17,511 filas, 20 columnas
Tabla cash_flow         creada: 17,511 filas, 22 columnas
```

2.1 SQLAlchemy: una forma de conectarse a cualquier base de datos

`create_engine()` de **SQLAlchemy** recibe una dirección (URL) que indica el tipo de base de datos y dónde está:

- SQLite: `sqlite:///financial_data.db`
- MySQL: `mysql+pymysql://usuario:contraseña@host:puerto/base` (`pymysql` es el "traductor" entre Python y MySQL).

A partir de ahí, `pd.read_sql()` funciona igual con las dos.

🔧 La contraseña de MySQL

Antes estaba escrita en el código (`DB_CONFIG['password']`). Si el archivo se sube a GitHub con la contraseña puesta, queda **pública**. Ahora se busca en la variable de entorno `MYSQL_PASSWORD` y, si no existe, se **pide al ejecutar** con `getpass` (que no muestra lo que escribes ni lo guarda en el notebook).

🔧 Las columnas en español también en SQLite

La base SQLite tiene las columnas originales en inglés (`stock`, `endDate`, `totalRevenue`…) y la de MySQL las tiene en español (las renombraste con `rename_columns_es.sql`). Para que **todo el código use los mismos nombres**, en SQLite se crean **vistas temporales** que renombran las columnas al vuelo:

- Una **vista** es una consulta guardada con nombre que se usa como si fuera una tabla.
- `TEMP` significa que solo existe mientras dura la conexión: **no modifica tu archivo `.db`**.

En MySQL no hace falta: se usan directamente tus tablas, que ya están en español.

```
# Equivalencia de columnas inglés (SQLite) → español (MySQL, rename_columns_es.sql)
COLUMNAS_ES = {
    'balance_sheet': {
        'stock': 'ticker', 'endDate': 'fecha_cierre',
        'totalCurrentAssets': 'activo_corriente_total', 'totalCurrentLiabilities':
'pasivo_corriente_total',
        'totalAssets': 'activo_total', 'totalLiab': 'pasivo_total',
        'totalStockholderEquity': 'patrimonio_neto_total', 'cash': 'efectivo', 'inventory':
'inventario',
        'netReceivables': 'cuentas_por_cobrar_netas', 'longTermDebt': 'deuda_largo_plazo',
        'accountsPayable': 'cuentas_por_pagar',
    },
    'income_statement': {
        'stock': 'ticker', 'endDate': 'fecha_cierre',
        'totalRevenue': 'ingresos_totales', 'netIncome': 'beneficio_neto', 'grossProfit':
'beneficio_bruto',
        'operatingIncome': 'beneficio_operativo', 'ebit': 'ebit', 'costOfRevenue': 'coste_ventas',
        'interestExpense': 'gastos_financieros',
    },
    'cash_flow': {
        'stock': 'ticker', 'endDate': 'fecha_cierre',
        'totalCashFromOperatingActivities': 'flujo_caja_operativo', 'capitalExpenditures':
'inversion_capex',
        'dividendsPaid': 'dividendos_pagados', 'netBorrowings': 'endeudamiento_neto',
    },
}


def conectar(fuente):
    # Devuelve una conexión abierta y el nombre de las 3 tablas (con columnas en español)
    if fuente == 'sqlite':
        if not RUTA_SQLITE.exists():
            raise FileNotFoundError(f'No se encuentra {RUTA_SQLITE}: ponlo en la carpeta del
notebook.')
        conexion = create_engine(f'sqlite:///{RUTA_SQLITE}').connect()
        tablas = {}
        for tabla, columnas in COLUMNAS_ES.items():
            seleccion = ', '.join(f'{en} AS {es}' for en, es in columnas.items())
```

```
            conexion.execute(text(f'CREATE TEMP VIEW {tabla}_es AS SELECT {seleccion} FROM
{tabla}'))
            tablas[tabla] = f'{tabla}_es'
        return conexion, tablas

    if fuente == 'mysql':
        contrasena = os.environ.get('MYSQL_PASSWORD') or getpass('Contraseña de MySQL: ')
        url = (f"mysql+pymysql://{MYSQL['user']}:{contrasena}"
               f"@{MYSQL['host']}:{MYSQL['port']}/{MYSQL['database']}")
        conexion = create_engine(url).connect()
        return conexion, {t: t for t in COLUMNAS_ES}

    raise ValueError("FUENTE debe ser 'sqlite' o 'mysql'")


conexion, TABLAS = conectar(FUENTE)
B, I, C = TABLAS['balance_sheet'], TABLAS['income_statement'], TABLAS['cash_flow']
print(f'Conectado a {FUENTE}. Tablas: {TABLAS}')
```

```
Conectado a sqlite. Tablas: {'balance_sheet': 'balance_sheet_es', 'income_statement':
'income_statement_es', 'cash_flow': 'cash_flow_es'}
```

```
def q(sql):
    # Ejecuta una consulta y devuelve el resultado como DataFrame
    return pd.read_sql(text(sql), conexion)


def anio(columna):
    # La función para sacar el año de una fecha cambia según la base de datos
    return f"CAST(strftime('%Y', {columna}) AS INTEGER)" if FUENTE == 'sqlite' else
f'YEAR({columna})'
```

`anio()` es un pequeño ayudante porque **cada base de datos tiene su dialecto**: SQLite saca el año con `strftime('%Y', fecha)` y MySQL con `YEAR(fecha)`. El resto de SQL de este notebook (`JOIN`, `GROUP BY`, `HAVING`, `CASE`, `LAG`, `NULLIF`…) es igual en las dos.

3. Exploración de los datos en SQL

(Antes: `exploracion_sql.ipynb`)

Antes de calcular nada, se explora en SQL, **directamente en la base de datos**. El SQL no es solo un "conector" para pasar datos a pandas: es una capa de trabajo real para revisar volumen, calidad y hacer los primeros agregados.

3.1 Volumen de cada tabla

Cuántas filas, cuántas empresas y qué fechas cubre cada estado. `UNION ALL` pega los resultados de varias consultas una debajo de otra (deben tener las mismas columnas).

```
q(f'''
SELECT 'balance_sheet' AS tabla, COUNT(*) AS filas, COUNT(DISTINCT ticker) AS empresas,
       MIN(fecha_cierre) AS fecha_min, MAX(fecha_cierre) AS fecha_max
FROM {B}
UNION ALL
SELECT 'income_statement', COUNT(*), COUNT(DISTINCT ticker), MIN(fecha_cierre), MAX(fecha_cierre)
FROM {I}
UNION ALL
SELECT 'cash_flow', COUNT(*), COUNT(DISTINCT ticker), MIN(fecha_cierre), MAX(fecha_cierre)
FROM {C}
''')
```

	tabla	filas	empresas	fecha_min	fecha_max
0	balance_sheet	17511	4422	2012-07-12	2020-05-02
1	income_statement	17511	4422	2012-07-12	2020-05-02
2	cash_flow	17511	4422	2012-07-12	2020-05-02

Con tus datos reales salían **17.511 filas y 4.422 empresas** en las tres tablas, de julio de 2012 a mayo de 2020.

3.2 Calidad: empresas con poco histórico

Una empresa con 1 o 2 años de datos sirve para su ficha de ese año, pero no para analizar **tendencias**. `HAVING` filtra **después de agrupar** (a diferencia de `WHERE`, que filtra filas antes de agrupar): aquí, las empresas con menos de 3 cierres.

```
pocos_anios = q(f'''
SELECT ticker, COUNT(DISTINCT fecha_cierre) AS anios_disponibles
FROM {B}
GROUP BY ticker
HAVING COUNT(DISTINCT fecha_cierre) < 3
ORDER BY anios_disponibles, ticker
''')
print(f'Empresas con menos de 3 años de histórico: {len(pocos_anios)}')
pocos_anios.head(10)
```

```
Empresas con menos de 3 años de histórico: 29
```

	ticker	anios_disponibles
0	ACTT	1
1	DFNS	1
2	HYAC	1
3	NFH	1
4	TMBR	1
5	APG	2
6	AYLA	2
7	BILL	2
8	ETNB	2
9	IMVT	2

En tus datos eran **29 de 4.422** empresas.

3.3 Agregados en SQL: rentabilidad por año

🔧 **Este era uno de los fallos.** La consulta original calculaba la **media** del ROE por año y en 2018 salía **468,1** (un 46.810 %), sin comentarlo. No es un error de la consulta: basta **una empresa con un patrimonio minúsculo** para que su ROE sea gigantesco y arrastre la media de miles de empresas.

Es exactamente el problema que el proyecto resuelve en el paso 6. Aquí se muestra junto a la **mediana**, que no se deja arrastrar por los extremos:

- **Media**: suma ÷ número de empresas. Un solo valor disparatado la descontrola.
- **Mediana**: el valor del medio si ordenas a todas las empresas. Los extremos no le afectan.

SQL no tiene una función `MEDIAN` común a SQLite y MySQL, así que la mediana se calcula con pandas a partir de los ROE de cada empresa, que sí se sacan con SQL.

```
roe_empresas = q(f'''
SELECT {anio('b.fecha_cierre')} AS anio,
       i.beneficio_neto * 1.0 / b.patrimonio_neto_total AS roe,
       i.beneficio_neto * 1.0 / b.activo_total AS roa
FROM {B} b
INNER JOIN {I} i ON b.ticker = i.ticker AND b.fecha_cierre = i.fecha_cierre
WHERE b.patrimonio_neto_total > 0
  AND b.activo_total > 0
  AND {anio('b.fecha_cierre')} BETWEEN 2016 AND 2019
''')

roe_por_anio = roe_empresas.groupby('anio').agg(
    empresas=('roe', 'size'),
    roe_media=('roe', 'mean'),
    roe_mediana=('roe', 'median'),
    roa_media=('roa', 'mean'),
    roa_mediana=('roa', 'median'),
).round(4)
roe_por_anio
```

anio	empresas	roe_media	roe_mediana	roa_media	roa_mediana
2016	3525	-0.2157	0.0645	-0.0473	0.0124
2017	3971	-0.3999	0.0649	-0.0232	0.0154
2018	4065	468.1352	0.0638	-0.0438	0.0145
2019	4083	-0.2197	0.0605	-0.0670	0.0133

Fíjate en la diferencia entre `roe_media` y `roe_mediana`: la mediana se mantiene en valores razonables (unos pocos puntos porcentuales) mientras la media puede dispararse en cualquier año. Por eso, en los agregados del paso 9 se usa la versión **winsorizada** de los ratios (se explica en el paso 6).

`* 1.0` convierte el número a decimal antes de dividir. En algunas bases de datos, dividir dos enteros da un entero (7 / 2 = 3), y así se evita.

3.4 Ranking: las 10 empresas que más facturan en 2019

🔧 En el original salían **GOOG y GOOGL**, que son **dos clases de acciones de la misma empresa** (Alphabet) con los mismos estados financieros: en un ranking de empresas ocupaba dos puestos. Aquí se buscan con SQL **todos** los cierres que comparten exactamente las mismas cifras con otro ticker:

```
duplicados = q(f'''
SELECT a.ticker AS ticker_1, b.ticker AS ticker_2, a.fecha_cierre, a.ingresos_totales
FROM {I} a
INNER JOIN {I} b
    ON  a.fecha_cierre = b.fecha_cierre
    AND a.ingresos_totales = b.ingresos_totales
    AND a.beneficio_neto = b.beneficio_neto
    AND a.ticker < b.ticker
WHERE a.ingresos_totales > 0
''')
print('Parejas de tickers con exactamente los mismos ingresos y beneficio en la misma fecha:')
duplicados
```

```
Parejas de tickers con exactamente los mismos ingresos y beneficio en la misma fecha:
```

	ticker_1	ticker_2	fecha_cierre	ingresos_totales
0	CHK	VEON	2017-12-31	9474000000
1	SHYF	SPAR	2019-12-31	756542000
2	SHYF	SPAR	2018-12-31	570527000
3	SHYF	SPAR	2017-12-31	404248000
4	SHYF	SPAR	2016-12-31	590777000
...	...	...	...	...
104	LILA	LILAK	2016-12-31	2723800000
105	CENT	CENTA	2019-09-28	2383010000
106	CENT	CENTA	2018-09-29	2215362000
107	CENT	CENTA	2017-09-30	2054478000
108	CENT	CENTA	2016-09-24	1829017000

109 rows × 4 columns

Es un **auto-*join***: la tabla se une consigo misma. `a.ticker < b.ticker` evita comparar un ticker consigo mismo y que cada pareja salga dos veces.

Con tus datos salen **109 parejas**, que afectan a **101 cierres de 27 tickers** (algunas empresas tienen tres clases de acciones). Son de tres tipos:

- **Varias clases de acciones de una misma empresa**: GOOG/GOOGL (Alphabet), FOX/FOXA, DISCA/DISCB/DISCK (Discovery), LBTYA/LBTYK (Liberty Global)...
- **Cambios de nombre o de ticker**: SPAR/SHYF (Spartan Motors pasó a llamarse The Shyft Group).
- **Errores del propio dataset**: por ejemplo, CHK y VEON con las mismas cifras en 2017, que son empresas distintas.

En los tres casos, en un ranking la misma cifra ocuparía dos puestos. Se excluye **el segundo ticker de cada pareja solo en el cierre duplicado** (no la empresa entera: VEON sigue en los años en que sus cifras son propias). Se guardan en `CIERRES_DUPLICADOS` para usarlos también en los gráficos del paso 9.

```
# Cierres a excluir de los rankings: el segundo ticker de cada pareja (p. ej. GOOGL; se conserva GOOG)
CIERRES_DUPLICADOS = (duplicados[['ticker_2', 'fecha_cierre']]
                      .rename(columns={'ticker_2': 'ticker'}).drop_duplicates())
CIERRES_DUPLICADOS['fecha_cierre'] = pd.to_datetime(CIERRES_DUPLICADOS['fecha_cierre'])
print(f'Cierres duplicados a excluir de los rankings: {len(CIERRES_DUPLICADOS)} '
      f'({CIERRES_DUPLICADOS["ticker"].nunique()} tickers)')

q(f'''
SELECT a.ticker, a.fecha_cierre, a.ingresos_totales
FROM {I} a
WHERE {anio('a.fecha_cierre')} = 2019
  AND NOT EXISTS (                      -- excluye el cierre si otro ticker "anterior" tiene las mismas cifras
      SELECT 1 FROM {I} b
      WHERE b.fecha_cierre = a.fecha_cierre
        AND b.ingresos_totales = a.ingresos_totales
        AND b.beneficio_neto = a.beneficio_neto
        AND b.ticker < a.ticker)
ORDER BY a.ingresos_totales DESC
LIMIT 10
''')
```

```
Cierres duplicados a excluir de los rankings: 101 (27 tickers)
```

	ticker	fecha_cierre	ingresos_totales
0	WMT	2019-01-31	514405000000
1	AMZN	2019-12-31	280522000000
2	BP	2019-12-31	276850000000
3	AAPL	2019-09-28	260174000000
4	CVS	2019-12-31	255765000000
5	XOM	2019-12-31	255583000000
6	UNH	2019-12-31	242155000000
7	MCK	2019-03-31	214319000000
8	T	2019-12-31	181193000000
9	ABC	2019-09-30	179589121000

`NOT EXISTS (subconsulta)` es verdadero cuando la subconsulta **no devuelve ninguna fila**. Aquí: "quédate con este cierre salvo que exista otro ticker, anterior en orden alfabético, con las mismas cifras en la misma fecha".

🔧 La consulta original filtraba `fecha_cierre = '2019-12-31'`, así que dejaba fuera a las empresas cuyo ejercicio no termina en diciembre. Ahora se filtra por **año** de cierre, y por eso aparecen **Walmart** (primera, con 514.405 millones de dólares), **Apple** y **McKesson**, que antes no salían. Ojo: Walmart cierra su ejercicio en enero, así que su cierre de enero de 2019 corresponde casi entero al año 2018.

3.5 Funciones de ventana: crecimiento de ingresos año a año

`LAG()` es una **función de ventana** (*window function*): mira el valor de **la fila anterior** sin agrupar ni perder filas.

- `PARTITION BY ticker`: la "fila anterior" se busca **dentro de la misma empresa**, nunca en otra.
- `ORDER BY fecha_cierre`: define qué es "anterior" (el año previo).
- `NULLIF(x, 0)` devuelve `NULL` si `x` es 0, para no dividir por cero.

```
q(f'''
SELECT
    ticker,
    fecha_cierre,
    ingresos_totales,
    LAG(ingresos_totales) OVER (PARTITION BY ticker ORDER BY fecha_cierre) AS
ingresos_anio_anterior,
    ROUND(
        (ingresos_totales - LAG(ingresos_totales) OVER (PARTITION BY ticker ORDER BY fecha_cierre))
* 100.0
        / NULLIF(LAG(ingresos_totales) OVER (PARTITION BY ticker ORDER BY fecha_cierre), 0),
        2) AS crecimiento_pct
FROM {I}
WHERE ticker IN ('AAPL', 'MSFT', 'AMZN', 'GOOG')
ORDER BY ticker, fecha_cierre
''')
```

	ticker	fecha_cierre	ingresos_totales	ingresos_anio_anterior	crecimiento_pct
0	AAPL	2016-09-24	215639000000	NaN	NaN
1	AAPL	2017-09-30	229234000000	2.156390e+11	6.30
2	AAPL	2018-09-29	265595000000	2.292340e+11	15.86
3	AAPL	2019-09-28	260174000000	2.655950e+11	-2.04
4	AMZN	2016-12-31	135987000000	NaN	NaN
5	AMZN	2017-12-31	177866000000	1.359870e+11	30.80
6	AMZN	2018-12-31	232887000000	1.778660e+11	30.93
7	AMZN	2019-12-31	280522000000	2.328870e+11	20.45
8	GOOG	2016-12-31	90272000000	NaN	NaN
9	GOOG	2017-12-31	110855000000	9.027200e+10	22.80
10	GOOG	2018-12-31	136819000000	1.108550e+11	23.42
11	GOOG	2019-12-31	161857000000	1.368190e+11	18.30
12	MSFT	2016-06-30	91154000000	NaN	NaN
13	MSFT	2017-06-30	96571000000	9.115400e+10	5.94
14	MSFT	2018-06-30	110360000000	9.657100e+10	14.28
15	MSFT	2019-06-30	125843000000	1.103600e+11	14.03

La primera fila de cada empresa sale vacía (`NaN`): no tiene año anterior con el que compararse. Con tus datos, Amazon crecía en torno al 30 % anual en 2017 y 2018.

3.6 Patrimonio neto negativo por año

`CASE WHEN ... THEN 1 ELSE 0 END` convierte una condición en 1 o 0, y al sumarlo se cuentan los casos que la cumplen.

🔧 La columna se llamaba `total_empresas`, pero cuenta **cierres** (empresa-año), no empresas. En tus datos, 2017 sumaba 4.425 "empresas" cuando solo hay 4.422 en total: algunas empresas **cambian de fecha de cierre** y tienen dos cierres dentro del mismo año natural. Se comprueba debajo.

```
q(f'''
SELECT {anio('fecha_cierre')} AS anio,
       COUNT(*) AS cierres,
       COUNT(DISTINCT ticker) AS empresas,
       SUM(CASE WHEN patrimonio_neto_total < 0 THEN 1 ELSE 0 END) AS con_patrimonio_negativo,
       ROUND(100.0 * SUM(CASE WHEN patrimonio_neto_total < 0 THEN 1 ELSE 0 END) / COUNT(*), 2) AS
pct
FROM {B}
WHERE {anio('fecha_cierre')} BETWEEN 2016 AND 2019
GROUP BY {anio('fecha_cierre')}
ORDER BY anio
''')
```

	anio	cierres	empresas	con_patrimonio_negativo	pct
0	2016	3947	3947	391	9.91
1	2017	4425	4388	414	9.36
2	2018	4436	4417	328	7.39
3	2019	4387	4379	261	5.95

```
q(f'''
SELECT ticker, {anio('fecha_cierre')} AS anio, COUNT(*) AS cierres_en_el_anio
FROM {B}
GROUP BY ticker, {anio('fecha_cierre')}
HAVING COUNT(*) > 1
''')
```

	ticker	anio	cierres_en_el_anio
0	ASM	2017	2
1	AUTL	2018	2
2	BBQ	2017	2
3	BGFV	2017	2
4	BJRI	2019	2
...	...	...	...
59	TILE	2017	2
60	TTMI	2018	2
61	UBS	2017	2
62	UFAB	2017	2
63	WEN	2017	2

64 rows × 3 columns

Si `cierres` es mayor que `empresas` en algún año, la diferencia son las empresas de esta última tabla. No es un error: el pipeline trabaja por **cierre** (empresa + fecha de cierre), así que cada cierre tiene su propia ficha.

4. Carga y unión de los 3 estados

(Antes: `load_joined_data()` en `indicators_sqlite.py` e `indicators_mysql.py`)

Se unen los tres estados por **empresa** (`ticker`) y **fecha de cierre** (`fecha_cierre`).

¿Por qué `INNER JOIN` ? Solo conserva los cierres que están **en las tres tablas**. Un indicador como el ROE necesita el beneficio (cuenta de resultados) y el patrimonio (balance); si falta un estado, ese cierre no puede tener la ficha completa. Con un `LEFT JOIN` aparecerían filas con huecos que darían ratios vacíos o engañosos.

```
def cargar_datos():
    consulta = f'''
    SELECT
        b.ticker, b.fecha_cierre,
        b.activo_corriente_total, b.pasivo_corriente_total, b.activo_total,
        b.pasivo_total, b.patrimonio_neto_total, b.efectivo, b.inventario,
        b.cuentas_por_cobrar_netas, b.deuda_largo_plazo, b.cuentas_por_pagar,
        i.ingresos_totales, i.beneficio_neto, i.beneficio_bruto,
        i.beneficio_operativo, i.ebit, i.coste_ventas, i.gastos_financieros,
        c.flujo_caja_operativo, c.inversion_capex, c.dividendos_pagados, c.endeudamiento_neto
    FROM {B} b
    INNER JOIN {I} i ON b.ticker = i.ticker AND b.fecha_cierre = i.fecha_cierre
    INNER JOIN {C} c ON b.ticker = c.ticker AND b.fecha_cierre = c.fecha_cierre
    '''
    df = q(consulta)
    df['fecha_cierre'] = pd.to_datetime(df['fecha_cierre'])
    # Ordenar por empresa y fecha: necesario para cualquier cálculo que compare con el año anterior
    return df.sort_values(['ticker', 'fecha_cierre']).reset_index(drop=True)


df = cargar_datos()
print(f'Filas tras el join: {len(df):,} | Empresas: {df["ticker"].nunique():,}')
df.head(3)
```

```
Filas tras el join: 17,511 | Empresas: 4,422
```

	ticker	fecha_cierre	activo_corriente_total	pasivo_corriente_total	activo_total	pasivo_total	patrimonio_neto_total	efectivo	inventario	cuentas_por_cobrar_netas	deuda_largo_plazo	cuentas_por_pagar	ingresos_totales	beneficio_neto	beneficio_bruto	beneficio_operativo	ebit	coste_ventas	gastos_financieros	flujo_caja_operativo	inversion_capex	dividendos_pagados	endeudamiento_neto
0	A	2016-10-31	3.635000e+09	9.450000e+08	7.794000e+09	3.548000e+09	4.243000e+09	2.289000e+09	533000000.0	7140000000.0	1.904000e+09	257000000.0	4202000000	462000000	2187000000	698000000	698000000	2005000000	-72000000.0	7.930000e+08	-1390000000.0	-1500000000.0	2620000000.0
1	A	2017-10-31	4.169000e+09	1.263000e+09	8.426000e+09	3.591000e+09	4.831000e+09	2.678000e+09	575000000.0	8010000000.0	1.801000e+09	305000000.0	4472000000	684000000	2399000000	853000000	853000000	2073000000	-79000000.0	8.890000e+08	-1760000000.0	-1700000000.0	1100000000.0
2	A	2018-10-31	3.848000e+09	1.171000e+09	8.541000e+09	3.970000e+09	4.567000e+09	2.247000e+09	638000000.0	8350000000.0	1.799000e+09	340000000.0	4914000000	316000000	2680000000	997000000	997000000	2234000000	-75000000.0	1.087000e+09	-1770000000.0	-1910000000.0	-210000000.0

```
# Comprobación: el join no debe duplicar cierres (una fila por empresa y fecha)
assert not df.duplicated(['ticker', 'fecha_cierre']).any(), 'Hay cierres duplicados tras el join'
print('✅ Una fila por empresa y fecha de cierre')
```

```
✅ Una fila por empresa y fecha de cierre
```

Signos de las partidas

Antes de usar las cifras hay que saber **con qué signo vienen**. En este dataset, los **gastos y las salidas de dinero vienen en negativo**:

```
for col in ['gastos_financieros', 'inversion_capex', 'dividendos_pagados']:
    valores = df[col].dropna()
    print(f'{col:22s}: {(valores < 0).mean():6.1%} negativos | {(valores > 0).mean():6.1%}
positivos')
```

```
gastos_financieros    :  99.9% negativos |   0.1% positivos
inversion_capex       : 100.0% negativos |   0.0% positivos
dividendos_pagados    : 100.0% negativos |   0.0% positivos
```

Esto condiciona dos fórmulas del paso 5:

- **Cobertura de intereses**: se usa el **valor absoluto** de los gastos financieros; si no, una empresa rentable saldría con cobertura negativa.
- **Flujo de caja libre**: como el CapEx ya es negativo, el flujo libre es flujo operativo **+** CapEx (que equivale a restar la inversión).

🔧 Vacíos frente a ceros: un fallo de la importación a MySQL

Un dato **vacío** ("la empresa no lo informó") no es lo mismo que un **cero** ("la empresa no invirtió nada"). Si un vacío se convierte en cero, los ratios salen con valores que parecen reales pero son falsos.

Comparando la ficha de Excel de la versión MySQL con los CSV originales, se ve que **al importar los CSV con el *Table Data Import Wizard* de MySQL Workbench, muchas celdas vacías se guardaron como 0**. En esa ficha:

Dato vacío en el CSV	Cierres	Qué salía en la ficha	Qué debería salir
Inversión (CapEx)	2.245	Flujo de caja libre = flujo operativo (como si no hubiera invertido nada)	Vacío
Flujo de caja operativo	303	Flujo de caja / beneficio = 0	Vacío
Activo corriente	11	Ratio de liquidez = 0	Vacío

La celda siguiente cuenta, para las columnas clave, cuántos valores hay **vacíos** y cuántos **a cero**. Con SQLite (creada desde los CSV en el paso 2.0) los vacíos se conservan. Si la ejecutas con `FUENTE = 'mysql'` y ves muchos ceros y casi ningún vacío en el CapEx, tu base MySQL tiene este problema: habría que volver a importar los CSV indicando que las celdas vacías son `NULL`.

```
calidad = pd.DataFrame({
    'vacíos': df[['activo_corriente_total', 'pasivo_corriente_total', 'pasivo_total',
'ingresos_totales',
                  'gastos_financieros', 'flujo_caja_operativo', 'inversion_capex']].isna().sum(),
    'ceros': (df[['activo_corriente_total', 'pasivo_corriente_total', 'pasivo_total',
'ingresos_totales',
                  'gastos_financieros', 'flujo_caja_operativo', 'inversion_capex']] == 0).sum(),
})
calidad
```

	vacíos	ceros
activo_corriente_total	180	0
pasivo_corriente_total	195	0
pasivo_total	185	0
ingresos_totales	0	983
gastos_financieros	4421	0
flujo_caja_operativo	304	0
inversion_capex	2245	0

Con tus CSV, el CapEx está vacío en 2.245 cierres y a cero en ninguno: no hay empresas que declaren "cero inversión", solo empresas que no lo informan. Por eso tratarlo como cero era un error. Los gastos financieros están vacíos en 4.421 cierres (una de cada cuatro empresas no los informa), y por eso la cobertura de intereses solo se puede calcular en unas 13.000.

En cambio, los **983 ingresos a cero** sí son ceros de verdad: empresas que todavía no venden nada (muchas biotecnológicas en fase de investigación). Para ellas los márgenes no se pueden calcular (dividir entre 0), y quedan vacíos.

5. Construcción de los indicadores

(Antes: `add_liquidity_ratios()`, `add_profitability_ratios()`, `add_solvency_ratios()` y `add_cash_quality_ratios()`)

🔧 Primero, una división segura

Muchos ratios dividen por partidas que pueden valer **0** (pasivo corriente, beneficio neto, ingresos...). En pandas, dividir por 0 da **infinito** (`inf`), no un error. Esos infinitos:

- hacían que `describe()` mostrara `inf` y `NaN` en las medias,
- **estropeaban el cálculo de percentiles** del paso 6, por eso la cobertura de intereses salía con un 0 % de valores extremos.

En la versión anterior, los infinitos se quitaban al final del pipeline completo, pero no en el paso a paso del notebook. Ahora la función `dividir()` devuelve **vacío** (`NaN`) cuando el denominador es 0, así que **ningún ratio puede salir infinito** en ningún momento.

```
def dividir(numerador, denominador):
    # División que devuelve NaN (vacío) en vez de infinito cuando el denominador es 0
    return numerador / denominador.replace(0, np.nan)


# Ejemplo
ejemplo = pd.DataFrame({'a': [10, 10, 10], 'b': [2, 0, np.nan]})
ejemplo['a / b normal'] = ejemplo['a'] / ejemplo['b']
ejemplo['dividir(a, b)'] = dividir(ejemplo['a'], ejemplo['b'])
ejemplo
```

	a	b	a / b normal	dividir(a, b)
0	10	2.0	5.0	5.0
1	10	0.0	inf	NaN
2	10	NaN	NaN	NaN

5.1 Liquidez: ¿puede pagar lo que debe a corto plazo?

Ratio	Fórmula	Cómo leerlo
`ratio_liquidez` (*current ratio*)	Activo corriente / Pasivo corriente	> 1: los activos a corto plazo cubren las deudas a corto plazo. Muy alto (> 3) puede indicar dinero o inventario ocioso: no siempre es "cuanto más, mejor".
`ratio_prueba_acida` (*quick ratio*)	(Activo corriente − Inventario) / Pasivo corriente	Más estricto: quita el inventario porque no siempre se puede vender rápido sin perder valor.

Si una empresa no informa de inventario (vacío), se trata como 0 (`fillna(0)`): muchas empresas de servicios no tienen inventario.

```
def ratios_liquidez(df):
    df = df.copy()
```

```
    df['ratio_liquidez'] = dividir(df['activo_corriente_total'], df['pasivo_corriente_total'])
    df['ratio_prueba_acida'] = dividir(df['activo_corriente_total'] - df['inventario'].fillna(0),
                                       df['pasivo_corriente_total'])
    return df


df = ratios_liquidez(df)
df[['ratio_liquidez', 'ratio_prueba_acida']].describe().round(3)
```

	ratio_liquidez	ratio_prueba_acida
count	17305.000	17305.000
mean	4.514	4.177
std	73.883	73.886
min	0.000	0.000
25%	0.951	0.736
50%	1.735	1.364
75%	3.248	2.710
max	8912.147	8912.147

5.2 Rentabilidad: ¿cuánto gana con lo que tiene y con lo que vende?

Ratio	Fórmula	Cómo leerlo
`roe` (*Return on Equity*)	Beneficio neto / Patrimonio neto	Rentabilidad para el accionista.
`roa` (*Return on Assets*)	Beneficio neto / Activo total	Rentabilidad de todos los activos, se financien como se financien. Más comparable entre empresas que el ROE.
`margen_neto`	Beneficio neto / Ingresos	Qué parte de cada euro vendido queda como beneficio final.
`margen_operativo`	Beneficio operativo / Ingresos	Rentabilidad del negocio principal, antes de intereses e impuestos.

⚠️ **El ROE con patrimonio negativo no tiene sentido.** Si una empresa **pierde dinero** y además tiene **patrimonio negativo**, el ROE sale **positivo** (negativo ÷ negativo), como si fuera rentable. Por eso no se borra, sino que se **marca** con `patrimonio_negativo` para poder excluirlo de las comparaciones.

```
def ratios_rentabilidad(df):
    df = df.copy()
    df['roe'] = dividir(df['beneficio_neto'], df['patrimonio_neto_total'])
    df['patrimonio_negativo'] = df['patrimonio_neto_total'] < 0
    df['roa'] = dividir(df['beneficio_neto'], df['activo_total'])
    df['margen_neto'] = dividir(df['beneficio_neto'], df['ingresos_totales'])
    df['margen_operativo'] = dividir(df['beneficio_operativo'], df['ingresos_totales'])
    return df


df = ratios_rentabilidad(df)
print(f'Cierres con patrimonio neto negativo: {df["patrimonio_negativo"].sum():,} '
      f'({df["patrimonio_negativo"].mean():.1%})')

engano = df[(df['beneficio_neto'] < 0) & df['patrimonio_negativo']]
print(f'De ellos, con pérdidas pero ROE positivo (engañoso): {len(engano):,}')
df[['ticker', 'fecha_cierre', 'roe', 'roa', 'margen_neto', 'margen_operativo']].head()
```

```
Cierres con patrimonio neto negativo: 1,410 (8.1%)
De ellos, con pérdidas pero ROE positivo (engañoso): 1,040
```

	ticker	fecha_cierre	roe	roa	margen_neto	margen_operativo
0	A	2016-10-31	0.108885	0.059276	0.109948	0.166111
1	A	2017-10-31	0.141586	0.081177	0.152952	0.190742
2	A	2018-10-31	0.069192	0.036998	0.064306	0.202890
3	A	2019-10-31	0.225569	0.113309	0.207438	0.202014
4	AA	2016-12-31	-0.070746	-0.023893	-0.042928	0.050655

En tus datos reales eran **1.410 cierres con patrimonio negativo (8,1 %)**.

5.3 Solvencia: ¿cuánto se endeuda y puede pagar los intereses?

Ratio	Fórmula	Cómo leerlo
`deuda_sobre_patrimonio` (*debt to equity*)	Pasivo total / Patrimonio neto	Cuánta deuda hay por cada euro de los dueños. Lo "normal" depende mucho del sector (un banco frente a una tecnológica).
`ratio_endeudamiento` (*debt ratio*)	Pasivo total / Activo total	Qué parte de los activos se financia con deuda.
`cobertura_intereses` (*interest coverage*)	EBIT / |Gastos financieros|	Cuántas veces el beneficio operativo cubre los intereses. Por debajo de 1,5-2 es señal de alerta.

🔧 Las **barras** `| |` significan **valor absoluto**. El código ya lo hacía bien (`.abs()`), pero el texto del notebook y la hoja de Metodología del Excel decían "EBIT / Gastos Financieros" sin el valor absoluto. Como los gastos financieros vienen en negativo, la fórmula escrita sin las barras daría lo contrario de lo que se quiere medir.

```python
def ratios_solvencia(df):
    df = df.copy()
    df['deuda_sobre_patrimonio'] = dividir(df['pasivo_total'], df['patrimonio_neto_total'])
    df['ratio_endeudamiento'] = dividir(df['pasivo_total'], df['activo_total'])
    # Los gastos financieros vienen en negativo (son un gasto): se usa el valor absoluto
    df['cobertura_intereses'] = dividir(df['ebit'], df['gastos_financieros'].abs())
    return df


df = ratios_solvencia(df)
df[['deuda_sobre_patrimonio', 'ratio_endeudamiento', 'cobertura_intereses']].describe().round(3)
```

	deuda_sobre_patrimonio	ratio_endeudamiento	cobertura_intereses
count	1.732600e+04	17315.000	13090.000
mean	2.437432e+04	0.871	4.556
std	2.296332e+06	27.691	8871.527
min	-2.361214e+03	0.000	-108673.408
25%	3.880000e-01	0.354	-0.778
50%	1.092000e+00	0.560	2.850
75%	2.734000e+00	0.783	8.437
max	2.445048e+08	3605.732	937568.312

5.4 Calidad del beneficio: ¿el beneficio se convierte en dinero real?

El beneficio contable puede no coincidir con el dinero que entra en caja (ventas que aún no se han cobrado, ajustes contables...). Un beneficio "en papel" que no se convierte en caja es una

señal de alerta clásica.

Indicador	Fórmula	Cómo leerlo
`flujo_caja_sobre_beneficio_neto`	Flujo de caja operativo / Beneficio neto	Idealmente > 1: la empresa convierte en caja todo su beneficio o más.
`flujo_caja_libre`	Flujo de caja operativo + CapEx	El dinero que queda tras invertir en activos fijos: lo que puede usar para pagar deuda, dividendos o recomprar acciones. No es un ratio, es un **importe**.

Recuerda (paso 4): el CapEx viene en negativo, así que **sumarlo equivale a restar la inversión**.

```
def ratios_calidad_beneficio(df):
    df = df.copy()
    df['flujo_caja_sobre_beneficio_neto'] = dividir(df['flujo_caja_operativo'],
df['beneficio_neto'])
    df['flujo_caja_libre'] = df['flujo_caja_operativo'] + df['inversion_capex']
    return df


df = ratios_calidad_beneficio(df)
df[['flujo_caja_sobre_beneficio_neto', 'flujo_caja_libre']].describe().round(3)
```

	flujo_caja_sobre_beneficio_neto	flujo_caja_libre
count	17203.000	1.526600e+04
mean	1.830	3.125918e+08
std	63.762	2.425954e+09
min	-2061.019	-1.160380e+11
25%	0.359	-1.239985e+07
50%	1.087	1.667800e+07
75%	1.908	1.542118e+08
max	5492.500	7.672600e+10

```
RATIOS = ['ratio_liquidez', 'ratio_prueba_acida', 'roe', 'roa', 'margen_neto', 'margen_operativo',
          'deuda_sobre_patrimonio', 'ratio_endeudamiento', 'cobertura_intereses',
          'flujo_caja_sobre_beneficio_neto']

# Comprobación: ningún ratio contiene infinitos
assert not np.isinf(df[RATIOS]).any().any(), 'Hay infinitos en los ratios'
print('✅ Ningún ratio contiene infinitos')
print('\nCierres sin dato en cada ratio (denominador 0 o dato contable vacío):')
print(df[RATIOS].isna().sum().to_string())
```

```
✅ Ningún ratio contiene infinitos

Cierres sin dato en cada ratio (denominador 0 o dato contable vacío):
ratio_liquidez                      206
ratio_prueba_acida                  206
roe                                 157
roa                                 172
margen_neto                         983
margen_operativo                    983
deuda_sobre_patrimonio              185
ratio_endeudamiento                 196
cobertura_intereses                4421
flujo_caja_sobre_beneficio_neto     308
```

6. Tratamiento de valores extremos

(Antes: `add_extreme_flags()` y `winsorize_ratios()`)

Con datos reales, algunos ratios se disparan cuando el denominador es casi cero (por ejemplo, un patrimonio de unos pocos miles de dólares en una empresa que factura millones). **No son errores del pipeline**: son situaciones reales que hay que **señalar, no esconder ni alterar**.

El criterio: dos versiones de los datos

Para...	Se usa	Qué pasa con los extremos
La **ficha de cada empresa**	El dato **original** + una columna `flag_extremo_<ratio>`	Se marcan, pero no se tocan: la ficha refleja la realidad contable
Medias, gráficos y comparativas	Una copia **winsorizada**	Se "topan" al percentil 1 % y 99 % para que no arrastren los agregados

Qué es un percentil: el percentil 99 % es el valor por debajo del cual está el 99 % de los datos. Todo lo que queda por encima del percentil 99 % o por debajo del 1 % se considera extremo.

Winsorizar (por el estadístico Charles Winsor) es sustituir los valores extremos por el valor del percentil límite: un ROE de 500 pasa a valer lo mismo que el percentil 99 %. Es distinto de **eliminar** la fila: la empresa sigue contando en la media, pero sin descontrolarla.

```
def marcar_extremos(df, inf=PERCENTIL_INF, sup=PERCENTIL_SUP):
    # Marca (sin modificar) los valores fuera de [percentil inf, percentil sup] de cada ratio
    df = df.copy()
    for col in RATIOS:
        lo, hi = df[col].quantile([inf, sup])
        df[f'flag_extremo_{col}'] = (df[col] < lo) | (df[col] > hi)
    return df


def winsorizar(df, inf=PERCENTIL_INF, sup=PERCENTIL_SUP):
    # Devuelve una COPIA con los ratios topados a los percentiles: solo para agregados y gráficos
    df = df.copy()
    for col in RATIOS:
        lo, hi = df[col].quantile([inf, sup])
        df[col] = df[col].clip(lower=lo, upper=hi)
    return df


df = marcar_extremos(df)
columnas_flag = [f'flag_extremo_{c}' for c in RATIOS]
print('% de cierres marcados como extremos en cada ratio:')
print((df[columnas_flag].mean() * 100).round(2).to_string())
print(f'\nCierres con ROE extremo: {df["flag_extremo_roe"].sum():,}')
```

```
% de cierres marcados como extremos en cada ratio:
flag_extremo_ratio_liquidez                    1.99
flag_extremo_ratio_prueba_acida                1.99
flag_extremo_roe                               1.99
flag_extremo_roa                               1.99
flag_extremo_margen_neto                       1.90
flag_extremo_margen_operativo                  1.90
flag_extremo_deuda_sobre_patrimonio            1.99
flag_extremo_ratio_endeudamiento               1.99
flag_extremo_cobertura_intereses               1.50
flag_extremo_flujo_caja_sobre_beneficio_neto   1.98

Cierres con ROE extremo: 348
```

Por definición, cada ratio marca un **2 %** de los cierres **que tienen dato** (el 1 % de abajo y el 1 % de arriba). El porcentaje de la tabla se calcula sobre **todos** los cierres, y los que no tienen dato nunca pueden ser extremos: por eso la cobertura de intereses, que falta en una de cada cuatro filas, marca un 1,5 % (el 2 % del 75 %), y los márgenes, que faltan en 983 filas, un 1,9 %.

🔧 En el notebook de MySQL, la cobertura de intereses salía con un **0,00 %** de extremos: los infinitos impedían calcular los percentiles. Ahora sale el 1,5 % esperado, igual que en el pipeline completo de la versión anterior.

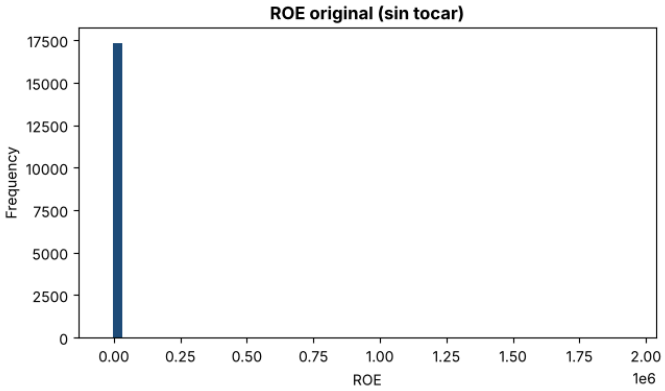
En tus datos salen **348 cierres con ROE extremo**, la misma cifra que en el proyecto original.

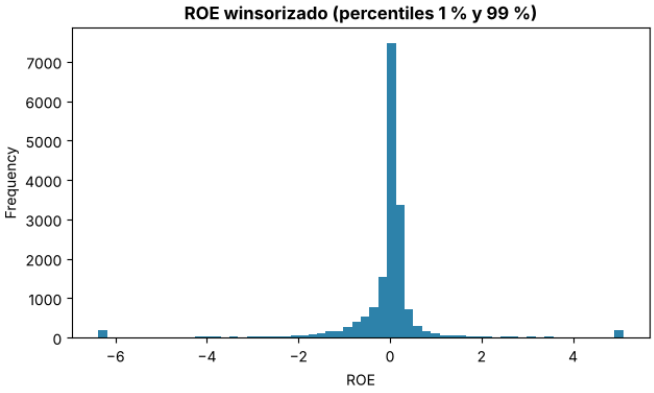
El efecto de winsorizar, en un gráfico

```
df_agregados = winsorizar(df)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
df['roe'].dropna().plot.hist(bins=60, ax=axes[0], color='#1F4E78')
axes[0].set_title('ROE original (sin tocar)', fontweight='bold')
axes[0].set_xlabel('ROE')
df_agregados['roe'].dropna().plot.hist(bins=60, ax=axes[1], color='#2E86AB')
axes[1].set_title('ROE winsorizado (percentiles 1 % y 99 %)', fontweight='bold')
axes[1].set_xlabel('ROE')
plt.tight_layout()
plt.show()

comparacion = pd.DataFrame({
    'original': df['roe'].describe(),
    'winsorizado': df_agregados['roe'].describe(),
}).round(3)
comparacion
```





	original	winsorizado
count	17354.000	17354.000
mean	110.041	-0.064
std	14721.968	1.125
min	-34756.774	-6.376
25%	-0.093	-0.093
50%	0.069	0.069
75%	0.150	0.150
max	1939064.700	5.098

A la izquierda, unos pocos valores enormes obligan a estirar el eje y la distribución real queda aplastada en una línea. A la derecha se ve la forma real de la distribución. Fíjate en la tabla: la **media** y la **desviación** cambian mucho, la **mediana** (50 %) casi nada.

🔧 En la versión anterior, el gráfico de la izquierda se "arreglaba" recortando los datos a [-2, 2] solo para dibujar. Aquí se muestra el original tal cual, para que se vea **por qué** hace falta winsorizar.

7. Pipeline completo y ficha financiera

(Antes: `build_financial_indicators()`)

Todo lo anterior se encadena en **una sola función**. Esto hace el proyecto **reproducible**: cualquier base de datos con las mismas 3 tablas pasa por el mismo proceso sin tocar el código.

Es el patrón típico de un **ETL** (*Extract, Transform, Load*): extraer (cargar y unir), transformar (calcular ratios y marcar extremos) y cargar (exportar, en el paso 8).

```
def construir_indicadores():
    df = cargar_datos()
    df = ratios_liquidez(df)
    df = ratios_rentabilidad(df)
    df = ratios_solvencia(df)
    df = ratios_calidad_beneficio(df)
    df = marcar_extremos(df)
    return df


ficha = construir_indicadores()

# Comprobación: el pipeline completo da exactamente lo mismo que el paso a paso
pd.testing.assert_frame_equal(ficha, df)
print(f'✅ Ficha financiera: {len(ficha):,} cierres, {ficha["ticker"].nunique():,} empresas, '
      f'{ficha.shape[1]} columnas')
ficha[['ticker', 'fecha_cierre', 'ratio_liquidez', 'roe', 'roa',
       'deuda_sobre_patrimonio', 'flujo_caja_libre']].head(10)
```

```
✅ Ficha financiera: 17,511 cierres, 4,422 empresas, 45 columnas
```

	ticker	fecha_cierre	ratio_liquidez	roe	roa	deuda_sobre_patrimonio	flujo_caja_libre
0	A	2016-10-31	3.846561	0.108885	0.059276	0.836201	6.540000e+08
1	A	2017-10-31	3.300871	0.141586	0.081177	0.743324	7.130000e+08
2	A	2018-10-31	3.286080	0.069192	0.036998	0.869280	9.100000e+08
3	A	2019-10-31	1.533173	0.225569	0.113309	0.990733	8.660000e+08
4	AA	2016-12-31	1.127614	-0.070746	-0.023893	1.599576	-7.150000e+08
5	AA	2017-12-31	1.303198	0.061685	0.015991	2.354411	8.190000e+08
6	AA	2018-12-31	1.482700	0.044500	0.015497	1.520826	4.900000e+07
7	AA	2019-12-31	1.377292	-0.273589	-0.076892	2.126702	3.070000e+08
8	AAL	2016-12-31	0.744233	0.682695	0.050396	12.546631	7.930000e+08
9	AAL	2017-12-31	0.595520	-1.643590	0.024287	-68.673077	-1.227000e+09

🔧 En la versión anterior, el paso a paso y el pipeline completo **no hacían exactamente lo mismo** (uno quitaba los infinitos y el otro no). Ahora la comprobación `assert_frame_equal` demuestra que dan resultados idénticos.

Consultar la ficha de una empresa

```
def ficha_empresa(ticker):
    columnas = ['fecha_cierre'] + RATIOS + ['flujo_caja_libre', 'patrimonio_negativo']
    return ficha.loc[ficha['ticker'] == ticker, columnas].set_index('fecha_cierre').T.round(3)
```

```
ficha_empresa('AAPL')
```

fecha_cierre	2016-09-24	2017-09-30	2018-09-29	2019-09-28
ratio_liquidez	1.352669	1.276063	1.132926	1.540126
ratio_prueba_acida	1.325684	1.227905	1.098802	1.501286
roe	0.356237	0.360702	0.555601	0.610645
roa	0.142024	0.128826	0.162775	0.16323
margen_neto	0.211868	0.210924	0.224142	0.212381
margen_operativo	0.278354	0.267604	0.26694	0.24572
deuda_sobre_patrimonio	1.508292	1.799906	2.413301	2.741004
ratio_endeudamiento	0.601322	0.642845	0.707029	0.732692
cobertura_intereses	41.225275	26.407232	21.882099	17.877517
flujo_caja_sobre_beneficio_neto	1.449668	1.328308	1.300734	1.255809
flujo_caja_libre	53497000000.0	51774000000.0	64121000000.0	58896000000.0
patrimonio_negativo	False	False	False	False

8. Exportación de la ficha a Excel

(Antes: `build_excel_sqlite.py` y `build_excel_mysql.py`)

El Excel tiene tres hojas:

1. **Metodología**: explica el proceso y cada fórmula (para quien abra el archivo sin ver el código).
2. **Indicadores**: la ficha de cada cierre con sus ratios y las marcas de valores extremos.
3. **Gráficos**: se añade en el paso 9.

Se usa **openpyxl**, la librería de Python para crear y dar formato a archivos Excel.

🔧 Correcciones:

- La ruta de salida es la carpeta `output/` del proyecto (antes, `/mnt/user-data/outputs/`).
- Los valores vacíos se escriben como **celdas vacías** (antes se escribía `nan`, que Excel no trata como número).
- La fórmula de la cobertura de intereses lleva el valor absoluto, igual que el código.
- Una sola función para las dos fuentes (antes había un archivo para SQLite y otro para MySQL).

```
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter

FUENTE_LETRA = 'Arial'
AZUL = '1F4E78'
RELLENO_CABECERA = PatternFill(start_color=AZUL, end_color=AZUL, fill_type='solid')
LETRA_CABECERA = Font(name=FUENTE_LETRA, bold=True, color='FFFFFF', size=10)
LETRA_CUERPO = Font(name=FUENTE_LETRA, size=10)
BORDE_INFERIOR = Border(bottom=Side(style='thin', color='D9D9D9'))

METODOLOGIA = [
    ('Fuente de datos', ''),
    ('', 'Kaggle: Financial Data of 4400+ Public Companies (balance, cuenta de resultados y flujo
de '
         'caja anuales, 2012-2020). Cargado en una base SQLite o MySQL con 3 tablas: '
         'balance_sheet, income_statement y cash_flow.'),
    ('', ''),
    ('Proceso (ETL)', ''),
    ('', '1. Unión de los 3 estados por empresa (ticker) y fecha de cierre (INNER JOIN).'),
    ('', '2. Cálculo de 4 bloques de indicadores. Las divisiones por 0 se dejan vacías, nunca
infinitas.'),
    ('', '3. Marcado de valores extremos por percentil, sin alterar el dato original.'),
    ('', ''),
    ('Indicadores', ''),
    ('Liquidez', 'ratio_liquidez = Activo corriente / Pasivo corriente'),
    ('', 'ratio_prueba_acida = (Activo corriente − Inventario) / Pasivo corriente'),
    ('Rentabilidad', 'roe = Beneficio neto / Patrimonio neto'),
    ('', 'roa = Beneficio neto / Activo total'),
    ('', 'margen_neto = Beneficio neto / Ingresos'),
    ('', 'margen_operativo = Beneficio operativo / Ingresos'),
    ('Solvencia', 'deuda_sobre_patrimonio = Pasivo total / Patrimonio neto'),
    ('', 'ratio_endeudamiento = Pasivo total / Activo total'),
```

```
    ('', 'cobertura_intereses = EBIT / |Gastos financieros|  (valor absoluto: los gastos vienen en
negativo)'),
    ('Calidad del beneficio', 'flujo_caja_sobre_beneficio_neto = Flujo de caja operativo /
Beneficio neto'),
    ('', 'flujo_caja_libre = Flujo de caja operativo + CapEx  (el CapEx viene en negativo)'),
    ('', ''),
    ('Criterio analítico', ''),
    ('', '• patrimonio_negativo: con patrimonio neto negativo el ROE pierde su sentido '
         '(pérdidas ÷ patrimonio negativo da un ROE positivo).'),
    ('', '• flag_extremo_<ratio>: valor fuera del rango [percentil 1 %, percentil 99 %] de su
ratio. '
         'No se elimina ni se modifica: solo se señala.'),
    ('', '• Para medias y gráficos se usa una copia winsorizada (valores topados a esos
percentiles).'),
    ('', '• Los rankings excluyen los cierres duplicados (mismas cifras con otro ticker, p. ej.
GOOGL frente a GOOG).'),
    ('', '• Un dato vacío en la fuente se mantiene vacío (no se trata como 0): el ratio que lo
necesita queda vacío.'),
    ('', '• La evolución anual se resume con la mediana, que representa a la empresa típica.'),
]


def hoja_metodologia(wb):
    ws = wb.active
    ws.title = 'Metodología'
    ws.sheet_view.showGridLines = False
    ws['B2'] = 'Ficha Financiera — Construcción de Indicadores desde Cero'
    ws['B2'].font = Font(name=FUENTE_LETRA, bold=True, size=14, color=AZUL)
    ws['B3'] = (f'Pipeline Python + {FUENTE.upper()} — 3 estados contables crudos → '
                f'{ficha["ticker"].nunique():,} empresas, {len(ficha):,} cierres')
    ws['B3'].font = Font(name=FUENTE_LETRA, italic=True, size=9, color='808080')

    fila = 5
    for etiqueta, texto in METODOLOGIA:
        if etiqueta and not texto:      # título de sección
            ws.cell(fila, 2, etiqueta).font = Font(name=FUENTE_LETRA, bold=True, size=11,
color=AZUL)
        else:
            if etiqueta:
                ws.cell(fila, 2, etiqueta).font = Font(name=FUENTE_LETRA, bold=True, size=10)
            if texto:
                ws.cell(fila, 3, texto).font = LETRA_CUERPO
        fila += 1
    ws.column_dimensions['A'].width = 3
    ws.column_dimensions['B'].width = 22
    ws.column_dimensions['C'].width = 110
```

```
COLUMNAS_FICHA = ['ticker', 'fecha_cierre', 'ingresos_totales', 'beneficio_neto', 'activo_total',
                  *RATIOS[:3], 'patrimonio_negativo', *RATIOS[3:], 'flujo_caja_libre',
*columnas_flag]
COLUMNAS_IMPORTE = {'ingresos_totales', 'beneficio_neto', 'activo_total', 'flujo_caja_libre'}


def hoja_indicadores(wb, ficha):
    ws = wb.create_sheet('Indicadores')
    ws.sheet_view.showGridLines = False

    datos = ficha[COLUMNAS_FICHA].copy()
    datos['fecha_cierre'] = datos['fecha_cierre'].dt.date
    # 🔧 NaN → None: openpyxl escribe None como celda vacía (NaN saldría como texto 'nan')
    datos = datos.astype(object).where(datos.notna(), None)
```

```
    ws.append(COLUMNAS_FICHA)
    for celda in ws[1]:
        celda.font, celda.fill = LETRA_CABECERA, RELLENO_CABECERA
        celda.alignment = Alignment(horizontal='center', vertical='center', wrap_text=True)

    for fila in datos.itertuples(index=False):
        ws.append(list(fila))

    # Formato de número por columna
    for j, nombre in enumerate(COLUMNAS_FICHA, start=1):
        if nombre in COLUMNAS_IMPORTE:
            formato = '$#,##0'
        elif nombre in RATIOS:
            formato = '0.00'
        elif nombre == 'fecha_cierre':
            formato = 'yyyy-mm-dd'
        else:
            formato = None
        for (celda,) in ws.iter_rows(min_row=2, min_col=j, max_col=j):
            celda.font, celda.border = LETRA_CUERPO, BORDE_INFERIOR
            if formato:
                celda.number_format = formato
        ws.column_dimensions[get_column_letter(j)].width = max(12, min(len(nombre) + 2, 30))

    ws.row_dimensions[1].height = 30
    ws.freeze_panes = 'C2'                        # deja fijas la cabecera y las columnas de empresa
y fecha
    ws.auto_filter.ref = ws.dimensions            # filtros en la cabecera
```

```
wb = Workbook()
hoja_metodologia(wb)
hoja_indicadores(wb, ficha)
wb.save(RUTA_EXCEL)
print(f'✅ Excel guardado en {RUTA_EXCEL} ({len(ficha):,} filas, {len(COLUMNAS_FICHA)} columnas)')
```

```
✅ Excel guardado en output/ficha_financiera_empresas.xlsx (17,511 filas, 27 columnas)
```

`freeze_panes = 'C2'` deja fijas la fila de cabecera y las dos primeras columnas (empresa y fecha) al desplazarte, y `auto_filter` añade los desplegables de filtro en la cabecera. Son detalles que hacen la ficha mucho más cómoda de usar.

9. Gráficos nativos en el Excel

(Antes: `build_charts_sqlite.py` y `build_charts_mysql.py`)

Se añaden gráficos **nativos** de Excel (no imágenes): quien abra el archivo puede cambiarles el formato y ver los datos de los que salen. Cada gráfico necesita sus datos escritos en una hoja, así que se crean dos hojas:

- **Datos_Gráficos**: las tablas de datos de cada gráfico.
- **Gráficos**: los gráficos.

Gráfico	Datos	Por qué esos datos
1. Evolución de la rentabilidad	🔧 **Medianas** por año de ROE, ROA y márgenes	Ni siquiera la media winsorizada sirve aquí (ver abajo)
2. Top 15 por ROE (2019)	ROE original	Sin patrimonio negativo, sin extremos y sin duplicados
3. Top 15 por flujo de caja libre (2019)	Flujo de caja libre, en millones	🔧 Sin tickers duplicados (antes Google podía salir dos veces)
4. Patrimonio positivo frente a negativo (2019)	Número de cierres	🔧 Colores asignados de verdad (antes el código para colorear no hacía nada)
5. ROA frente a endeudamiento (2019)	Muestra de 300 cierres, winsorizados	Para ver la relación sin saturar el gráfico
6. Distribución del ratio de liquidez (2019)	Cierres por tramos	Muestra cuántas empresas están por debajo de 1

```
from openpyxl.chart import BarChart, LineChart, PieChart, ScatterChart, Reference, Series
from openpyxl.chart.label import DataLabelList
from openpyxl.chart.marker import Marker
from openpyxl.chart.series import DataPoint

PALETA = ['1F4E78', '2E86AB', '5DA9E9', 'F2A541', 'E76F51', '6A994E', '9B5DE5']


def etiquetas_datos(valor=False, porcentaje=False):
    # Etiquetas de datos: se indica explícitamente qué mostrar (si no, algunos programas muestran
todo)
    etiquetas = DataLabelList()
    etiquetas.showVal, etiquetas.showPercent = valor, porcentaje
    etiquetas.showSerName = etiquetas.showCatName = etiquetas.showLegendKey = False
    return etiquetas


def escribir_tabla(ws, fila, col, cabeceras, filas):
    # Escribe una tabla en la hoja de datos y devuelve la última fila usada
    for j, h in enumerate(cabeceras):
        celda = ws.cell(fila, col + j, h)
        celda.font, celda.fill = LETRA_CABECERA, RELLENO_CABECERA
    for i, valores in enumerate(filas, start=1):
        for j, v in enumerate(valores):
            ws.cell(fila + i, col + j, v).font = LETRA_CUERPO
    return fila + len(filas)


# Datos de partida para todos los gráficos
ficha['anio'] = ficha['fecha_cierre'].dt.year
ficha_agregados = winsorizar(ficha)
# Marca los cierres duplicados detectados en el paso 3.4 (se excluyen de los rankings)
```

```
claves = ficha[['ticker', 'fecha_cierre']].merge(CIERRES_DUPLICADOS, how='left', indicator=True)
sin_duplicados = (claves['_merge'] == 'left_only').values
f2019 = ficha[(ficha['anio'] == 2019) & sin_duplicados]
a2019 = ficha_agregados[(ficha_agregados['anio'] == 2019) & sin_duplicados]
print(f'Cierres de 2019: {(ficha["anio"] == 2019).sum():,} | sin duplicados: {len(f2019):,}')
```

```
Cierres de 2019: 4,387 | sin duplicados: 4,362
```

Preparar los datos de cada gráfico

Se calculan primero todas las tablas (así puedes verlas aquí en el notebook) y después se dibujan los gráficos.

```
# 1. Evolución de la rentabilidad: mediana por año (ver explicación debajo)
ratios_tendencia = ['roe', 'roa', 'margen_neto', 'margen_operativo']
f_tendencia = ficha[ficha['anio'].between(2016, 2019) & ~ficha['patrimonio_negativo']]
tendencia = f_tendencia.groupby('anio')[ratios_tendencia].median().round(4).reset_index()

media_winsorizada = (ficha_agregados[ficha_agregados['anio'].between(2016, 2019)]
                     .groupby('anio')[ratios_tendencia].mean().round(4))
print('Media winsorizada (lo que mostraba el gráfico original):')
display(media_winsorizada)
print('Mediana (lo que muestra ahora):')
display(tendencia.set_index('anio'))

# 2. Top 15 por ROE en 2019 (sin patrimonio negativo ni ROE extremo)
top_roe = (f2019[~f2019['patrimonio_negativo'] & ~f2019['flag_extremo_roe']]
           .nlargest(15, 'roe')[['ticker', 'roe']].round(3))

# 3. Top 15 por flujo de caja libre en 2019, en millones
top_fcl = f2019.nlargest(15, 'flujo_caja_libre')[['ticker', 'flujo_caja_libre']].copy()
top_fcl['fcl_millones'] = (top_fcl['flujo_caja_libre'] / 1e6).round(1)

# 4. Patrimonio positivo frente a negativo en 2019
reparto_patrimonio = (f2019['patrimonio_negativo']
                      .map({False: 'Patrimonio positivo', True: 'Patrimonio negativo'})
                      .value_counts().reindex(['Patrimonio positivo', 'Patrimonio negativo'],
fill_value=0)
                      .reset_index())
reparto_patrimonio.columns = ['categoria', 'cierres']

# 5. ROA frente a endeudamiento: muestra de 300 cierres de 2019
dispersion = a2019[['ratio_endeudamiento', 'roa']].dropna()
dispersion = dispersion.sample(n=min(300, len(dispersion)),
random_state=42).sort_values('ratio_endeudamiento').round(4)

# 6. Distribución del ratio de liquidez por tramos
tramos = [0, 0.5, 1, 1.5, 2, 3, 5, np.inf]
etiquetas = ['<0.5', '0.5-1', '1-1.5', '1.5-2', '2-3', '3-5', '>5']
distribucion = (pd.cut(f2019['ratio_liquidez'].dropna(), bins=tramos, labels=etiquetas,
right=False)
                .value_counts().reindex(etiquetas).reset_index())
distribucion.columns = ['tramo', 'cierres']

display(tendencia, top_roe.head(), top_fcl.head(), reparto_patrimonio, distribucion)
```

```
Media winsorizada (lo que mostraba el gráfico original):
```

anio	roe	roa	margen_neto	margen_operativo
2016	-0.0328	-0.1275	-1.9787	-1.8128
2017	-0.0145	-0.1130	-1.7855	-1.6405
2018	-0.0798	-0.1068	-1.9197	-1.6820
2019	-0.1306	-0.1037	-1.7202	-1.6052

```
Mediana (lo que muestra ahora):
```

anio	roe	roa	margen_neto	margen_operativo
2016	0.0645	0.0124	0.0542	0.1116
2017	0.0649	0.0154	0.0550	0.1011
2018	0.0638	0.0145	0.0540	0.1059
2019	0.0601	0.0133	0.0513	0.0962

	anio	roe	roa	margen_neto	margen_operativo
0	2016	0.0645	0.0124	0.0542	0.1116
1	2017	0.0649	0.0154	0.0550	0.1011
2	2018	0.0638	0.0145	0.0540	0.1059
3	2019	0.0601	0.0133	0.0513	0.0962

	ticker	roe
14550	SPGI	4.432
7973	IHRT	3.847
11729	ORLY	3.501
2462	BSIG	3.477
16091	UI	3.250

	ticker	flujo_caja_libre	fcl_millones
35	AAPL	5.889600e+10	58896.0
10302	MS	3.894700e+10	38947.0
10334	MSFT	3.826000e+10	38260.0
6881	GOOG	3.097200e+10	30972.0
15085	T	2.903300e+10	29033.0

	categoria	cierres
0	Patrimonio positivo	4101
1	Patrimonio negativo	261

	tramo	cierres
0	<0.5	619
1	0.5-1	562
2	1-1.5	746
3	1.5-2	590
4	2-3	629
5	3-5	530
6	>5	635

🔧 **El gráfico de evolución usaba la media winsorizada**, y con tus datos el **margen neto medio sale −172 % a −198 %** y el **ROA medio −10 % a −13 %**, como si la empresa típica perdiera dinero a espuertas. No es así: la mediana del margen neto ronda el +5 %. Pasa porque hay muchas empresas jóvenes (sobre todo biotecnológicas) que casi no venden y pierden mucho: su margen es de −1.000 % o peor. Son tantas que el propio percentil 1 % ya es enorme, y winsorizar no basta. La **mediana** representa a la empresa típica; además se excluyen los cierres con patrimonio negativo, en los que el ROE no tiene sentido.

Otros dos detalles corregidos aquí:

- En el gráfico 5, la muestra de 300 se calculaba **antes** de quitar los vacíos (`dropna`). Si quedaban menos de 300 filas, `sample()` daba error. Ahora se quitan los vacíos primero.

- En el gráfico 6, `right=False` hace que cada tramo **incluya su límite inferior**: una empresa con ratio exactamente 1 cuenta en "1-1.5" y no en "0.5-1". Así "<0.5" significa de verdad "menor que 0,5".

Dibujar los gráficos en el Excel

```
wb = load_workbook(RUTA_EXCEL)
for nombre in ['Datos_Gráficos', 'Gráficos']:      # si ya existían de una ejecución anterior, se
rehacen
    if nombre in wb.sheetnames:
        del wb[nombre]
ws_datos = wb.create_sheet('Datos_Gráficos')
ws_graf = wb.create_sheet('Gráficos')
for ws in (ws_datos, ws_graf):
    ws.sheet_view.showGridLines = False
ws_graf.sheet_properties.tabColor = 'F2A541'
ws_graf['A1'] = 'Gráficos — Ficha Financiera'
ws_graf['A1'].font = Font(name=FUENTE_LETRA, bold=True, size=14, color=AZUL)

# --- 1. Líneas: evolución de la rentabilidad ---
ult = escribir_tabla(ws_datos, 2, 1, ['Año', 'ROE mediano', 'ROA mediano', 'Margen neto mediano',
                                      'Margen operativo mediano'], tendencia.values.tolist())
g1 = LineChart()
g1.title, g1.height, g1.width = 'Evolución de la rentabilidad mediana (2016-2019)', 9, 16
g1.y_axis.title, g1.x_axis.title = 'Ratio', 'Año'
g1.add_data(Reference(ws_datos, min_col=2, max_col=5, min_row=2, max_row=ult),
titles_from_data=True)
g1.set_categories(Reference(ws_datos, min_col=1, min_row=3, max_row=ult))
for i, serie in enumerate(g1.series):
    serie.graphicalProperties.line.solidFill = PALETA[i]
    serie.graphicalProperties.line.width = 25000
    serie.smooth = False                          # líneas rectas entre años (no curvas)
ws_graf.add_chart(g1, 'B3')

# --- 2. Barras: top 15 ROE ---
ult = escribir_tabla(ws_datos, 2, 7, ['Empresa', 'ROE'], top_roe.values.tolist())
g2 = BarChart()
g2.type, g2.title, g2.height, g2.width = 'bar', 'Top 15 empresas por ROE (2019)', 10, 16
g2.add_data(Reference(ws_datos, min_col=8, min_row=2, max_row=ult), titles_from_data=True)
g2.set_categories(Reference(ws_datos, min_col=7, min_row=3, max_row=ult))
g2.series[0].graphicalProperties.solidFill = PALETA[1]
g2.legend = None
g2.dataLabels = etiquetas_datos(valor=True)
ws_graf.add_chart(g2, 'B23')

# --- 3. Barras: top 15 flujo de caja libre ---
ult = escribir_tabla(ws_datos, 2, 11, ['Empresa', 'Flujo de caja libre', 'FCL (millones $)'],
                     top_fcl.values.tolist())
g3 = BarChart()
g3.type, g3.title, g3.height, g3.width = 'bar', 'Top 15 empresas por flujo de caja libre, 2019
(millones $)', 10, 16
g3.add_data(Reference(ws_datos, min_col=13, min_row=2, max_row=ult), titles_from_data=True)
g3.set_categories(Reference(ws_datos, min_col=11, min_row=3, max_row=ult))
g3.series[0].graphicalProperties.solidFill = PALETA[4]
g3.legend = None
ws_graf.add_chart(g3, 'K23')

# --- 4. Donut: patrimonio positivo frente a negativo ---
ult = escribir_tabla(ws_datos, 25, 1, ['Categoría', 'Cierres'], reparto_patrimonio.values.tolist())
g4 = PieChart()
g4.title, g4.height, g4.width = 'Patrimonio neto positivo frente a negativo (2019)', 9, 10
```

```python
g4.add_data(Reference(ws_datos, min_col=2, min_row=25, max_row=ult), titles_from_data=True)
g4.set_categories(Reference(ws_datos, min_col=1, min_row=26, max_row=ult))
for idx, color in enumerate([PALETA[5], PALETA[4]]):        # 🔧 colores asignados de verdad
    punto = DataPoint(idx=idx)
    punto.graphicalProperties.solidFill = color
    g4.series[0].dPt.append(punto)
g4.dataLabels = etiquetas_datos(porcentaje=True)
ws_graf.add_chart(g4, 'B43')

# --- 5. Dispersión: ROA frente a endeudamiento ---
ult = escribir_tabla(ws_datos, 2, 16, ['Ratio endeudamiento', 'ROA'], dispersion.values.tolist())
g5 = ScatterChart()
g5.title, g5.height, g5.width = 'ROA frente a endeudamiento (muestra 2019)', 10, 16
g5.x_axis.title, g5.y_axis.title = 'Ratio de endeudamiento', 'ROA'
serie = Series(Reference(ws_datos, min_col=17, min_row=2, max_row=ult),
               Reference(ws_datos, min_col=16, min_row=3, max_row=ult), title_from_data=True)
serie.marker = Marker(symbol='circle', size=5)
serie.marker.graphicalProperties.solidFill = PALETA[2]
serie.marker.graphicalProperties.line.noFill = True
serie.graphicalProperties.line.noFill = True
g5.series.append(serie)
g5.legend = None
ws_graf.add_chart(g5, 'K3')

# --- 6. Columnas: distribución del ratio de liquidez ---
ult = escribir_tabla(ws_datos, 2, 20, ['Tramo ratio de liquidez', 'Cierres'],
distribucion.values.tolist())
g6 = BarChart()
g6.type, g6.title, g6.height, g6.width = 'col', 'Distribución del ratio de liquidez (2019)', 10, 16
g6.y_axis.title, g6.x_axis.title = 'Nº de cierres', 'Tramo'
g6.add_data(Reference(ws_datos, min_col=21, min_row=2, max_row=ult), titles_from_data=True)
g6.set_categories(Reference(ws_datos, min_col=20, min_row=3, max_row=ult))
g6.series[0].graphicalProperties.solidFill = PALETA[3]
g6.legend = None
g6.dataLabels = etiquetas_datos(valor=True)
ws_graf.add_chart(g6, 'K43')

for j in range(1, 22):
    ws_datos.column_dimensions[get_column_letter(j)].width = 18

wb.save(RUTA_EXCEL)
print(f'✅ Gráficos añadidos. Hojas del Excel: {load_workbook(RUTA_EXCEL,
read_only=True).sheetnames}')
```

```
✅ Gráficos añadidos. Hojas del Excel: ['Metodología', 'Indicadores', 'Datos_Gráficos', 'Gráficos']
```

Cómo funciona un gráfico de openpyxl:

1. Se escriben los datos en una hoja (`escribir_tabla`).
2. `Reference(...)` señala el rango de celdas con los datos y otro con las categorías (los nombres del eje).
3. `add_data(..., titles_from_data=True)` usa la primera fila del rango como nombre de la serie (por eso los rangos de datos empiezan en la fila de cabecera y los de categorías una fila más abajo).
4. `add_chart(grafico, 'B3')` lo coloca con la esquina superior izquierda en esa celda.

🔧 Además de los colores del donut:

- Antes, `ws.column_dimensions[chr(64 + col)]` calculaba la letra de la columna a mano (`chr(65)` = 'A'), y fallaba a partir de la columna 27. `get_column_letter()` lo hace bien

con cualquier columna.

- Si ejecutas la celda dos veces, las hojas de gráficos **se rehacen** en vez de duplicarse.

```
conexion.close()
print('Conexión cerrada')
```

```
Conexión cerrada
```

10. Conclusiones

Sobre el proceso

- Construir los indicadores desde los estados crudos obliga a **decidir el criterio de cada fórmula** (qué denominador, qué signo, qué hacer con el patrimonio negativo) en vez de heredar un ratio sin cuestionarlo.
- **Separar el dato individual de la vista agregada** (marcar en la ficha, winsorizar en los agregados) es una decisión de negocio, no solo técnica: la ficha de cada empresa refleja su realidad contable, y las medias no se dejan arrastrar por unos pocos casos.
- La misma lógica se resuelve en **SQL** (exploración, agregados, funciones de ventana) y en **Python** (pipeline y exportación), usando cada herramienta para lo que mejor hace.

Sobre los datos (con tus resultados reales)

- 4.422 empresas y 17.511 cierres con los 3 estados disponibles.
- Un dato vacío no es un cero: tratarlos igual inflaba el flujo de caja libre de 2.245 cierres en la versión MySQL.
- El 8,1 % de los cierres tiene **patrimonio neto negativo**: en ellos el ROE no es interpretable.
- La **media** de ratios como el ROE no sirve sin tratar los extremos (el ROE medio de 2018 salía un 46.810 %); la **mediana** o la versión **winsorizada** sí.

Cómo reutilizar el pipeline

- Para pasar de SQLite a MySQL basta con cambiar `FUENTE` en el paso 1.
- Cualquier base de datos con las mismas 3 tablas pasa por `construir_indicadores()` sin tocar el código, también los datos **trimestrales** del mismo dataset.

Resumen de correcciones

Problema en la versión anterior	Solución
9 archivos con código duplicado (SQLite y MySQL)	Un solo notebook; la fuente se elige con `FUENTE`
Los notebooks de SQLite importaban `indicators` (no existe)	El código está en el propio notebook
Rutas `/mnt/user-data/outputs/` (no existen en Windows)	Carpeta `output/` relativa al notebook
Contraseña de MySQL dentro del código	Variable de entorno o `getpass` al ejecutar
Infinitos en las tablas y percentiles mal calculados en el paso a paso	`dividir()`: las divisiones por 0 quedan vacías desde el principio
Cobertura de intereses con 0 % de extremos	Corregido al eliminar los infinitos
El paso a paso y el pipeline completo no daban lo mismo	Comprobado con `assert_frame_equal`
Media del ROE de 2018 = 468 sin comentar	Media frente a mediana, y explicación
GOOG y GOOGL en los rankings (101 cierres duplicados en total, 27 tickers)	Detección con un auto-*join* y exclusión con `NOT EXISTS`
Importación a MySQL: vacíos convertidos en 0 (2.245 flujos de caja libre inflados)	Tabla de vacíos frente a ceros y aviso para reimportar con `NULL`
Gráfico de evolución con la media winsorizada (margen neto medio de −190 %)	Mediana por año
Ranking filtrando `'2019-12-31'` (excluía a Apple, Microsoft…)	Filtro por año de cierre
"total_empresas" contaba cierres	Columnas `cierres` y `empresas`, y detección de dobles cierres
Fórmula de cobertura sin valor absoluto en los textos	Texto coherente con el código: EBIT / |Gastos financieros|
`nan` escrito en el Excel	Celdas vacías
Colores del donut sin aplicar	`DataPoint` con color
Muestra del gráfico de dispersión antes de quitar vacíos	`dropna()` antes de `sample()`
Letras de columna con `chr()`	`get_column_letter()`
Hojas de gráficos duplicadas al ejecutar dos veces	Se borran y se rehacen